

EMS Project — Service Discovery & Load Balancing

Learning Notes and Experiment Record

Eureka • Spring Cloud LoadBalancer • API Gateway • Multiple Instances • Failover

1. Purpose

The objective was to replace fixed backend addressing with service discovery and client-side load balancing. The experiment was incremental: register services with Eureka, make the API Gateway discover EMS-BACKEND, run multiple backend instances, and then stop one instance to observe availability.

2. Current Architecture

```
React UI → API Gateway (:8082) → Spring Cloud LoadBalancer → EMS-BACKEND instances
discovered through Eureka
```

Eureka Server: :8761

API Gateway: :8082

EMS-BACKEND instances: :8080 and :8084

3. Service Discovery

Eureka acts as the service registry. Both backend processes register under the same logical service name, **EMS-BACKEND**, while listening on different ports.

```
EMS-BACKEND → instance :8080
EMS-BACKEND → instance :8084
```

The backend logs show successful Eureka registration for both ports. The 8080 instance registered as EMS-BACKEND on port 8080, and the second instance registered as EMS-BACKEND on port 8084.

4. Gateway and Logical Routing

The Employee route uses:

```
uri=lb://EMS-BACKEND
```

This is different from a fixed URI such as `http://localhost:8080`. The **lb://** scheme tells Spring Cloud Gateway to resolve the logical service through Spring Cloud LoadBalancer.

Gateway logs also showed the `ReactiveLoadBalancerClientFilter` in the filter chain and successful HTTP 200 responses for requests routed through **lb://EMS-BACKEND**.

5. Load-Balancing Experiment

Step	Observed result
Start backend :8080	Registered with Eureka as EMS-BACKEND.
Start backend :8084	Second instance registered with Eureka as EMS-BACKEND.
Call Employee API through Gateway	Requests used the logical EMS-BACKEND route.
Inspect backend logs	EmployeeController requests appeared on backend instances.
Stop :8084	Application continued working through the remaining healthy instance :8080.

6. What We Learned from Instance Failure

Stopping the 8084 instance did not make the application unavailable. Requests continued through the remaining 8080 instance.

```
Before failure:  
LoadBalancer → { EMS-BACKEND:8080, EMS-BACKEND:8084 }  
  
After 8084 becomes unavailable:  
LoadBalancer → { EMS-BACKEND:8080 }
```

This demonstrates the basic availability benefit of service discovery plus client-side load balancing: the Gateway does not need a manual route change when one backend instance becomes unavailable.

7. What This Demonstrates — and What It Does Not

Capability	Status
Service Discovery	Demonstrated with Eureka.
Service Registration	8080 and 8084 registered as EMS-BACKEND.
Client-side Load Balancing	Gateway uses <code>lb://EMS-BACKEND</code> .
Multiple Service Instances	Demonstrated.
Basic Failure Handling	Stopping 8084 still allowed traffic through 8080.
Circuit Breaker	Not implemented; this is a separate resilience mechanism.
Application-level Fallback	Not implemented.
Exact 50/50 distribution	Not established; logs prove traffic reached both instances.

8. Key Concepts to Remember

- **Eureka** is the service registry. Services register themselves and discover services by logical name.
- **EMS-BACKEND** is the logical service identity; :8080 and :8084 are separate runtime instances.
- **lb://EMS-BACKEND** tells the Gateway to resolve the service through load balancing instead of a hard-coded port.
- **Spring Cloud LoadBalancer** selects an available service instance for the Gateway request.
- Multiple instances improve availability because another instance can continue serving when one instance becomes unavailable.
- Service discovery/load balancing is not the same as a circuit breaker.

9. Production Considerations Not Yet Implemented

- Circuit breaker and fallback behavior.
- Deliberate timeout and retry policies.
- Health/readiness-aware traffic management.
- Correlation IDs and distributed tracing.
- Centralized structured logging and log redaction.
- Observability dashboards and service-level metrics.

10. Important Logging Observation

The Gateway DEBUG logs included a complete JWT Authorization header. JWTs are sensitive credentials and should not be exposed in production logs or shared publicly. Logging should later be reduced or configured to prevent authentication tokens from appearing in logs.

11. Current Learning Milestone

```
Eureka Server
↓
Service Registration
↓
Service Discovery
↓
Gateway → lb://EMS-BACKEND
↓
Multiple Backend Instances
↓
Load Balancing
↓
Instance Failure Test
↓
Traffic continues through the healthy instance
```

12. Recommended Next Learning Step

The next useful experiment is to distinguish **service-discovery failure detection** from **request-time failure handling**. Observe what happens when a backend instance fails while a request is being made, then use that observation to introduce timeouts, retries, and eventually a circuit breaker.

Conclusion: The EMS project has successfully demonstrated Eureka-based service discovery, Gateway-based client-side load balancing, multiple backend instances, and continued application availability after one backend instance was stopped.